

app.js

Cài thư viện

```
// 1.npm init -y
// 2.npm install nodemon
// 3.npm install express
// 4.npm install ejs
```

Tóm tắt hoạt động chuẩn

Giai đoạn	Hành vi	Diễn giải
/	Hiển thị danh sách users	Dữ liệu render từ mảng <code>users</code>
/create + POST	Tạo user mới	Thêm id tăng dần
/update/:id (GET)	Mở form update	Truyền user tương ứng vào EJS
/update/:id (POST)	Nhận dữ liệu form	Cập nhật user trong mảng
/delete/:id	Xóa user	Filter bỏ phần tử đó

```
//imp thu vien
const express = require("express")
const path = require("path")
const app = express()
```

1. `const express = require("express")`

- `require()` : là hàm của Node.js dùng để *nạp (import)* một thư viện (module) bên ngoài vào file hiện tại.
- `"express"` : là tên thư viện mà bạn muốn nạp. Express là một *framework (khuôn mẫu)* giúp tạo server web nhanh và dễ hơn so với chỉ dùng Node.js thuần.
- `const express = ...` : gán thư viện Express vào biến `express` , để sau này bạn có thể dùng các hàm của Express như `express()` , `express.Router()` , v.v.

 *Hiểu đơn giản:* Dòng này giúp bạn "kéo" thư viện Express vào để dùng cho ứng dụng web Node.js của mình.

2. `const path = require("path")`

- `path` là module có sẵn (built-in) của Node.js, không cần cài thêm.
- Nó dùng để làm việc với **đường dẫn file/thư mục**: nối đường dẫn, lấy tên file, định dạng lại path sao cho tương thích với mọi hệ điều hành (Windows, macOS, Linux).

Ví dụ:

```
path.join(__dirname, "views", "index.ejs")
```

→ sẽ tự động tạo ra đường dẫn chính xác `D:\project\views\index.ejs` (hoặc tương đương trên Linux).

 **Hiểu đơn giản:** Dòng này giúp bạn xử lý đường dẫn file/thư mục trong dự án an toàn và tiện lợi hơn.

3. `const app = express()`

- Khi bạn gọi `express()`, nó sẽ **tạo ra một ứng dụng web (web app instance)**.
- Biến `app` chính là **server Express** mà bạn sẽ dùng để:
 - định nghĩa route: `app.get("/", (req, res) => {...})`
 - cấu hình middleware: `app.use(express.static("public"))`
 - lắng nghe port: `app.listen(3000)`

 **Hiểu đơn giản:** `app` chính là "trái tim" của ứng dụng Express – nơi bạn cài đặt mọi thứ để server hoạt động.

Tổng kết ý nghĩa đoạn code:

```
// Import thư viện
const express = require("express") // nạp Express framework
const path = require("path")       // nạp module xử lý đường dẫn
const app = express()              // khởi tạo ứng dụng Express
```

 **Kết quả:** Sau ba dòng này, bạn đã có một "khung xương" để bắt đầu viết server Node.js bằng Express.

Từ đây bạn có thể tiếp tục thêm route, render view, hoặc kết nối database.

1. Middleware chuyển dữ liệu form thành đối tượng

```
app.use(express.urlencoded({ extended: true }))
```

Giải thích:

- `app.use()` dùng để **gắn middleware** (trình trung gian) cho toàn bộ ứng dụng.
- `express.urlencoded()` là middleware có sẵn trong Express giúp **phân tích dữ liệu gửi từ form HTML (method POST)**.

- Khi người dùng submit form, dữ liệu sẽ gửi dạng `application/x-www-form-urlencoded` (vd: `name=iPhone&price=1000`).
- Middleware này sẽ **chuyển chuỗi đó thành đối tượng JavaScript** để bạn có thể truy cập bằng `req.body` .

 Ví dụ:

```
<form method="POST" action="/create">
  <input name="name" value="iPhone" />
</form>
```

→ `req.body` trong server sẽ là `{ name: "iPhone" }`

2. Dữ liệu mẫu (fake data)

```
let phones = [
  { id: 1, name: "iPhone 15 Pro", brand: "Apple", price: 1000 },
  ...
];
```

Giải thích:

- Đây là mảng chứa 5 sản phẩm mẫu (giống như database tạm).
- Mỗi phần tử là một **object (đối tượng)** có `id` , `name` , `brand` , `price` .

 Khi chưa có database (MySQL, MongoDB...), ta thường dùng mảng này để test logic.

3. Khai báo thư mục chứa view và engine

```
app.set("views", path.join(__dirname, "views"));
app.set("view engine", "ejs")
```

Giải thích:

- `views` : chỉ định thư mục chứa các file hiển thị giao diện (EJS).
- `__dirname` : là đường dẫn thư mục hiện tại.
- `path.join(__dirname, "views")` : nối thành đường dẫn tuyệt đối như `D:\project\views` .
- `view engine` : đặt công cụ render view, ở đây là `"ejs"` .

 → Khi gọi `res.render("read")` , Express sẽ tự hiểu file cần hiển thị là

`views/read.ejs` .

4. Route hiển thị danh sách (READ)

```
app.get("/", (req, res) => {  
  res.render("read", { phones })  
})
```

Giải thích:

- `app.get("/")` : định nghĩa route cho trang chủ.
- `res.render("read", { phones })` : render file `views/read.ejs` và truyền biến `phones` để hiển thị danh sách.

 → Trong `read.ejs`, bạn có thể lặp qua `phones` bằng:

```
<% phones.forEach(p => { %>  
  <tr><td><%= p.name %></td></tr>  
<% }) %>
```

+ 5. Route tạo mới (CREATE)

```
app.get("/create", (req, res) => {  
  res.render("create")  
})  
app.post("/create", (req, res) => {  
  const data = req.body;  
  const newPhone = {  
    ...data,  
    id: phones[phones.length - 1].id + 1  
  }  
  phones.push(newPhone)  
  res.redirect("/")  
});
```

Giải thích:

- `GET /create` : hiển thị form thêm mới (`create.ejs`)
- `POST /create` : nhận dữ liệu form gửi lên qua `req.body`
- `phones.push(newPhone)` : thêm sản phẩm mới vào mảng
- `res.redirect("/")` : sau khi thêm, quay lại trang danh sách

 `...data` (spread operator): sao chép toàn bộ dữ liệu trong `req.body`.

6. Route cập nhật (UPDATE)

```
app.get("/update/:id", (req, res) => {
  const { id } = req.params;
  let phone = phones.find(item => item.id === id)
  res.render("update", { phone })
})
app.post("/update/:id", (req, res) => {
  const { id } = req.params;
  const data = req.body;
  const index = phones.findIndex(item => item.id === id);
  if (index !== -1) {
    phones[index] = { ...phones[index], ...data, id: Number(id) };
  }
  res.redirect("/")
});
```

Giải thích:

- `:id` là **tham số động** trong URL (vd: `/update/3`)
- `req.params.id` : lấy giá trị `3`
- `phones.find(...)` : tìm sản phẩm có id tương ứng
- `res.render("update", { phone })` : hiển thị form update, điền sẵn dữ liệu cũ
- Khi submit, cập nhật dữ liệu trong mảng và quay lại danh sách

 Mẹo: `findIndex()` trả về vị trí phần tử trong mảng → giúp bạn thay thế dữ liệu cũ.

7. Route xóa (DELETE)

```
app.get("/delete/:id", (req, res) => {
  const { id } = req.params;
  phones = phones.filter(item => item.id !== id)
  res.redirect("/")
});
```

Giải thích:

- `filter()` tạo ra mảng mới loại bỏ phần tử có id trùng với id cần xóa.
- Sau khi xóa xong → `res.redirect("/")` quay lại danh sách.

📖 Dạng "DELETE bằng GET" này chỉ dùng tạm khi học, trong thực tế nên dùng `DELETE` method hoặc xác nhận qua form.

🚀 8. Khởi động server

```
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
})
```

🔍 Giải thích:

- `app.listen(PORT, callback)` → mở cổng 3000 để server chạy.
- `http://localhost:3000` là địa chỉ bạn mở trong trình duyệt.

📖 Mỗi khi bạn chỉnh sửa code, nên restart server (hoặc dùng `nodemon` để tự restart).

✅ Tóm tắt hoạt động tổng thể

Chức năng	HTTP Method	Đường dẫn	View	Mô tả
Read	GET	<code>/</code>	<code>read.ejs</code>	Hiển thị danh sách điện thoại
Create	GET / POST	<code>/create</code>	<code>create.ejs</code>	Hiển thị form và thêm mới
Update	GET / POST	<code>/update/:id</code>	<code>update.ejs</code>	Cập nhật điện thoại theo ID
Delete	GET	<code>/delete/:id</code>	—	Xóa điện thoại theo ID